Commit0 Automation Pipeline

Automated GitHub Repository Discovery, Filtering, and Dataset Generation System

Technical Documentation

VERSION 1.0 MARCH 2026 CONFIDENTIAL

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Pipeline Stages
4. Performance Optimizations
5. Storage Management
6. Error Handling & Resilience
7. Logging & Monitoring
8. Configuration Reference
9. Prerequisites
10. Output Files

1. Executive Summary

`commit0_automation.py` is a fully automated, production-grade Python pipeline designed to discover, evaluate, and curate high-quality open-source Python repositories for the **Commit0 Benchmark**. The system discovers up to **10,000 repositories** from GitHub using a star-range partitioning strategy, applies a rigorous **two-stage filtering process** (11 quality checks + 6 benchmark validation checks), and for every qualifying repository generates structured **YAML metadata** and **PDF documentation** (via headless Playwright/Chromium rendering). All artifacts are automatically uploaded to **Google Drive** in an organized, date-stamped folder structure. The entire pipeline — from virtual environment creation to Drive upload — executes with **zero user interaction**, self-bootstraps its dependencies, and leverages multi-threaded parallelism across all stages for a **~3.5× end-to-end speedup** compared to sequential execution.

10,000	17	8	~1.5h
MAX REPOS DISCOVERED	QUALITY CHECKS	PARALLEL PHASES	1K REPOS PIPELINE

> **Key Differentiator:** The script is entirely self-contained in a single 2,643-line Python file. It auto-creates its virtual environment, installs all dependencies, provisions a headless browser, handles GitHub API rate limits, and resumes gracefully from interruptions — no external orchestrator (Airflow, Celery, etc.) required.

2. Architecture Overview

The pipeline is structured as a sequential orchestration of parallelized stages, with a retry loop around the discovery–filtering phase to guarantee a minimum viable dataset.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          COMMIT0 AUTOMATION PIPELINE                     │
└──────────────────────────────────────────────────────────────────────────┘

  ┌──────────────────────┐
  │  PHASE 0             │
  │  Self-Bootstrap      │   Auto-create venv, install deps, install Chromium
  │  (one-time setup)    │   Re-execute inside venv
  └──────────────────────┘
            │
            ▼
  ┌──────────────────────┐
  │  Load State          │   Read processed_repos.json + raw_repos.jsonl
  │  Seed Exclusions     │   Build excluded_names set (already-seen repos)
  └──────────────────────┘
            │
            ▼
  ┌ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┐
  │ DISCOVERY RETRY LOOP  (up to 20 rounds)                                │
  │                                                                        │
  │  ┌──────────────────────┐                                              │
  │  │  STEP 1: Discovery   │  Star-range partitioning (20 ranges)         │
  │  │  (GitHub Search API) │   Paginated fetch, dedup, exclusion          │
  │  └──────────────────────┘                                              │
  │            │                                                           │
  │            ▼                                                           │
  │  ┌──────────────────────┐                                              │
  │  │  STEP 2: Filter 1    │  11 quality checks                           │
  │  │  ▓▓▓▓▓▓▓▓ 8 workers   │  ~11% pass rate                              │
  │  └──────────────────────┘                                              │
  │            │                                                           │
  │            ▼                                                           │
  │  ┌──────────────────────┐                                              │
  │  │  STEP 3: Filter 2    │  Commit0 benchmark validation                │
  │  │  ▓▓▓▓▓▓▓▓ 8 workers   │  ~77% pass rate (of Filter 1)                │
  │  └──────────────────────┘                                              │
  │            │                                                           │
  │            ├── < MIN_FILTER2_PASS? ──→ Clear caches, retry ─┐          │
  │            │                                                │          │
  │            └── ≥ MIN_FILTER2_PASS ──→ Proceed ──────────────┐│          │
  │                                                             │└──────────┘
  └ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ─ ┘
            ▼
  ┌──────────────────────┐
  │  Setup build_dataset │   Clone reference repo, patch upstream
  └──────────────────────┘   bugs (scrape_pdf.py)
            ▼
  ┌──────────────────────┐
  │  PHASE 1: Clone      │   git clone --depth 1
  │  ▓▓▓▓▓▓▓▓ 8 workers   │   Shallow clones, parallel
  └──────────────────────┘
            ▼
  ┌──────────────────────┐
  │  PHASE 2: YAML Gen   │   Auto-detect: src_dir, test_cmd, python
  │  ▓▓▓▓▓▓ 6 workers     │   version, install cmd, docs URL, tag/SHA
  └──────────────────────┘
            ▼
  ┌──────────────────────┐
  │  PHASE 3: PDF Gen    │   Direct RTD download → Playwright BFS
  │  ▓▓▓ 3 workers        │   crawl (max 30 pages) → single A4 PDF
  └──────────────────────┘   → bz2 compress → delete raw
            ▼
```

```
┌──────────────────────┐
│ PHASE 4: Cleanup     │  Delete cloned repos to free disk
│                      │  5.9 GB → ~350 MB savings
└──────────────────────┘
          │
          ▼
┌──────────────────────┐
│ repos.yml Append     │  Smart index management, deterministic
└──────────────────────┘
          │
          ▼
┌──────────────────────┐
│ Google Drive Upload  │  OAuth2, date-stamped folders
│ ▓▓▓▓▓ 5 workers      │  Parallel file uploads
└──────────────────────┘
          │
          ▼
┌──────────────────────┐
│ Summary Tables       │  Pipeline steps, final counts,
│ 🥂 Complete 🥂       │  per-repo status (Clone/YAML/PDF/BZ2/Drive)
└──────────────────────┘
```

3. Pipeline Stages

3.1 Phase 0: Self-Bootstrapping

The script is designed to run from a bare Python installation with zero pre-configuration. On first execution, Phase 0:

1. Creates a dedicated virtual environment at `.venv_commit0/`
2. Installs all required packages via pip: `requests`, `PyYAML`, `PyMuPDF`, `PyPDF2`, `beautifulsoup4`, `playwright`, `google-api-python-client`, `google-auth-oauthlib`
3. Installs the Playwright Chromium browser binary
4. Re-executes itself inside the venv via `os.execv()` — fully transparent to the caller

> On subsequent runs, the venv already exists and the script proceeds directly to the pipeline. No manual `pip install` or environment activation is ever required.

3.2 Step 1: Repository Discovery

The discovery engine uses a **star-range partitioning strategy** to work around GitHub's Search API limitation of 1,000 results per query. Instead of a single broad search, it divides the star count space into 20 non-overlapping ranges:

Range	Stars	Range	Stars
1	≥ 100,000	11	3,500 – 3,999
2	50,000 – 99,999	12	3,000 – 3,499
3	30,000 – 49,999	13	2,500 – 2,999
4	20,000 – 29,999	14	2,000 – 2,499
5	15,000 – 19,999	15	1,500 – 1,999
6	10,000 – 14,999	16	1,200 – 1,499
7	8,000 – 9,999	17	1,000 – 1,199
8	6,000 – 7,999	18	800 – 999
9	5,000 – 5,999	19	600 – 799
10	4,000 – 4,999	20	500 – 599

Each range fetches up to 10 pages of 100 results, with deduplication via a global `seen` set. Previously discovered repos (loaded from `raw_repos.jsonl`) and previously processed repos (from `processed_repos.json`) are excluded, ensuring re-runs always discover *new* repositories. Output is saved as JSONL (one JSON object per line) for streaming-friendly processing.

3.3 Step 2: Filter 1 — Initial Quality Checks

All discovered repos are evaluated against 11 quality checks, executed in parallel across **8 worker threads**. A repo must pass *all* checks to proceed.

#	Check	Type	Criteria
1	Not a Fork	MUST	Rejects forked repositories
2	Not Archived	MUST	Rejects archived/read-only repos
3	Not ML/AI Framework	MUST	Blocklist: tensorflow, pytorch, keras, torch, jax, huggingface, transformers, llm, diffusion, etc.
4	Not CLI Tool	MUST	Blocklist: cli, command-line, terminal, console, shell-tool
5	Not Native Wrapper	MUST	Blocklist: binding, wrapper, ffi, ctypes, cffi, pybind11, cython, pyo3, swig
6	Repo Size	MUST	≤ 500 MB (500,000 KB)
7	Star Count	MUST	≥ 3,000 stars (fallback threshold)
8	Python Ratio	MUST	≥ 95% Python by bytes (via GitHub Languages API)
9	No Native Extensions	MUST	No ext_modules, Extension(), cythonize() in setup.py/cfg; no .c/.cpp/.pyx/.so files
10	Documentation Website	MUST	Has homepage URL, ReadTheDocs, GitHub Pages, or docs/ directory
11	Project Structure	MUST	Has pyproject.toml/setup.py + tests/ directory + src/ or package directory with __init__.py

Typical pass rate: ~11% (e.g., 109 out of 1,000 repos)

3.4 Step 3: Filter 2 — Commit0 Benchmark Validation

Filter 1 passers are further validated against benchmark-specific criteria using **8 parallel worker threads**.

Check	Type	Description
Uses pytest	MUST	conftest.py, pytest.ini, [tool.pytest] in config, or pytest in requirements
No GPU Usage	MUST	No cuda/cupy/nvidia/gpu keywords in requirements or config files
Installable	SHOULD	pip install -e . viable (pyproject.toml with [build-system] or setup.py)
Stars ≥ 5,000	SHOULD	Higher star count indicates maturity and maintenance
Proper Structure	SHOULD	Has standard project layout (revalidated from Filter 1 context)
Python 3.10+	SHOULD	requires-python compatible with modern Python

A repo must pass *all* MUST checks and score ≥ 2 out of 4 SHOULD checks. Additional heuristics (dependency count < 100, test isolation) are evaluated but don't block acceptance.

Typical pass rate: ~77% of Filter 1 passers (e.g., 84 out of 109)

3.5 Discovery Retry Loop

If fewer than `MIN_FILTER2_PASS` (default: 3) repos pass both filters, the pipeline:

1. Clears intermediate cache files (filtered_stage1.json, final_selected.json)
2. Adds all discovered repos to the exclusion set
3. Restarts discovery with fresh, never-before-seen repositories
4. Repeats up to `MAX_RETRY_ROUNDS` (default: 20) times

This guarantees the pipeline always produces a minimum viable dataset, even if initial discovery rounds yield low-quality results.

3.6 Phase 1: Parallel Clone

Accepted repositories are cloned using **8 parallel worker threads**. Each clone uses `git clone --depth 1` (shallow clone), which downloads only the latest commit — reducing download size by approximately 80% compared to full history.

Failed clones are logged and gracefully excluded from subsequent phases. The pipeline continues processing all successfully cloned repos.

3.7 Phase 2: Parallel YAML Generation

For each cloned repository, the system auto-detects all metadata fields via GitHub API inspection and local file analysis, running across **6 parallel worker threads**:

Field	Detection Method
`name`	Repository full_name (owner/repo)
`commit` / `tag`	Latest release tag via Releases API; falls back to latest commit SHA
`python`	Parsed from `requires-python` in pyproject.toml/setup.py/setup.cfg
`install`	Detects optional-dependencies extras (dev/test/all) for `pip install -e .[extra]`
`packages`	Scans for requirements.txt and variants (dev, test)
`specification`	Documentation URL: homepage, ReadTheDocs, GitHub Pages
`test_cmd`	Determines pytest invocation (checks for benchmark plugins, custom addopts)
`test_dir`	Detects tests/ or test/ directory
`src_dir`	Identifies src/ layout or package directory with __init__.py

Each repo's metadata is written to both an individual `.yml` file and appended to the combined `repos.yml` with deterministic index numbering.

3.8 Phase 3: Parallel PDF Documentation Generation

PDF generation runs across **3 parallel Playwright instances** (limited due to memory requirements of headless Chromium). The process follows a two-tier strategy:

Tier 1: Direct Download (Fast Path)

For ReadTheDocs-hosted projects, the system first attempts to download pre-built PDFs from standardized RTD URLs (`/_/downloads/en/latest/pdf/` and `/stable/pdf/`). If successful, this completes in seconds.

Tier 2: Playwright BFS Crawl (Fallback)

When direct download is unavailable, the system launches a full documentation crawl:

1. **BFS Crawl:** Starting from the documentation homepage, explores up to `MAX_CRAWL_PAGES` (30) pages, following same-domain links
2. **Smart URL Filtering:** Skips non-content paths (`/_sources/` , `/_static/` , `/_modules/` , `/genindex` , etc.) and binary files (.zip, .tar, .whl)
3. **Content Deduplication:** Hashes first 2,000 characters of each page's content to eliminate duplicate pages
4. **Content Extraction:** Removes navigation, sidebars, footers, and headers; extracts main content using priority-ordered CSS selectors (`[role='main']` , `main` , `article` , `.document` , etc.)
5. **Combined HTML:** All pages are merged into a single HTML document with professional CSS styling (proper typography, code blocks, tables, page breaks)
6. **Single PDF Render:** Playwright renders the combined HTML as a clean A4 PDF with proper margins (20mm top/bottom, 15mm sides)

After PDF generation, the raw file is **bz2-compressed** and the original PDF is deleted to save storage. Output structure: `build_dataset/pdfs/{repo_name}/{repo_name}.pdf.bz2`

3.9 Phase 4: Disk Cleanup

After all phases complete, cloned repositories are deleted via `shutil.rmtree()` . Combined with raw PDF deletion after bz2 compression, this reduces storage from **5.9 GB to ~350 MB** for 84 repos (a **94% reduction**).

3.10 Google Drive Upload

The final stage uploads all artifacts to Google Drive using **5 parallel upload threads**:

- **Authentication:** OAuth2 flow — opens browser for first-time authorization, caches token at `drive_token.json` for automatic refresh on subsequent runs
- **Folder Structure:** `Automation Scripts Data / {DD Month YYYY} / PDF/ + Raw Data/` — date-stamped for each run
- **Uploads:** raw_repos.jsonl → Raw Data/; per-repo PDF subdirectories → PDF/

4. Performance Optimizations

The pipeline has been extensively optimized for throughput. Every I/O-bound stage runs in parallel using `concurrent.futures.ThreadPoolExecutor`, and unnecessary delays have been eliminated or minimized.

4.1 Parallelization Summary

Optimization	Before	After	Speedup
Filter 1 (API checks)	Sequential + `sleep(0.5)` per check	`ThreadPoolExecutor(8)`, zero sleep	**~6–8×**
Filter 2 (validation)	Sequential + `sleep(0.3)` per check	`ThreadPoolExecutor(8)`, zero sleep	**~6–8×**
Git Clone	Sequential	`ThreadPoolExecutor(8)`	**~8×**
YAML Generation	Sequential	`ThreadPoolExecutor(6)`	**~6×**
PDF Generation	Sequential	`ThreadPoolExecutor(3)`	**~3×**
Drive Upload	Sequential, one file at a time	`ThreadPoolExecutor(5)`	**~5×**
HTTP Connections	New TCP connection per request	`requests.Session` with connection pool (20 connections)	**~30%**
Rate Limiting	Fixed `sleep()` everywhere	Dynamic based on `X-RateLimit-Remaining` header	**Eliminates wasted sleep**
Inter-stage Delays	`sleep(2/3/5)` between operations	`sleep(0.5/1/2)` — just enough	**~60% less idle**

4.2 Estimated Pipeline Time (1,000 repos)

Stage	Before Optimization	After Optimization
Discovery	~2 min	~1.5 min
Filter 1	~30 min	~5 min
Filter 2	~15 min	~3 min
Clone (~84 repos)	~20 min	~3 min
YAML (~84 repos)	~15 min	~3 min
PDF (~84 repos)	~4 hours	~1.5 hours
Drive Upload	~30 min	~8 min
TOTAL	**~5.5 hours**	**~1.5–2 hours**

4.3 Connection Pooling

A shared, thread-safe `requests.Session` with `HTTPAdapter(pool_connections=20, pool_maxsize=20)` eliminates TCP handshake overhead. The session is lazily initialized with double-checked locking for thread safety.

4.4 Smart Rate Limiting

Instead of fixed sleep intervals, the `github_get()` function reads the `X-RateLimit-Remaining` response header after each request. If remaining requests drop below 100, a brief 1-second delay is introduced; below 50 triggers a 2-second delay. This prevents hitting the hard rate limit wall while minimizing unnecessary idle time.

5. Storage Management

Storage optimization is critical for large-scale runs. At 10,000 repos, naive storage would consume ~620 GB. The pipeline implements four strategies to reduce this by **97%**:

Strategy	Description	Savings
Shallow Clone	`git clone --depth 1` downloads only the latest commit, excluding full history	~80% per clone
Post-Processing Cleanup	Phase 4 deletes all cloned repos after YAML + PDF generation is complete	100% of clone storage
PDF → BZ2 Compression	Raw PDF is bz2-compressed; original deleted after compression	~50–70% per PDF
Selective Crawling	MAX_CRAWL_PAGES = 30 limits doc site crawling to essential pages	Prevents unbounded growth

5.9 GB
BEFORE (84 REPOS)

350 MB
AFTER (84 REPOS)

94%
REDUCTION

~20 GB
PROJECTED (10K REPOS)

6. Error Handling & Resilience

The pipeline is designed for unattended, long-running execution. Every failure mode is handled gracefully to ensure maximum throughput even under adverse conditions.

Failure Scenario	Handling
GitHub API rate limit (403)	Reads `X-RateLimit-Reset` header, sleeps until reset + 5s buffer, then retries
GitHub server error (5xx)	Exponential backoff retry ($2^{attempt}$ seconds), up to 3 attempts
Authentication failure (401)	Removes Authorization header and retries without token (falls back to anonymous access)
Network timeout / DNS failure	Caught per-request with configurable timeout (30s API, 60s downloads); logged and skipped
Clone failure	Repo excluded from Phase 2/3; other repos continue processing
YAML generation failure	Repo excluded from Phase 3; other repos continue processing
PDF generation failure	Logged as SKIPPED; other repos continue. Repo still gets YAML entry
Drive upload failure	Wrapped in try/except — pipeline completes successfully even if upload fails entirely
Missing credentials.json	FileNotFoundError with clear instructions for obtaining OAuth2 credentials from GCP Console
Pipeline interruption (kill/crash)	Resume via `processed_repos.json` — re-run skips already-processed repos and discovers new ones
Insufficient Filter 2 results	Retry loop discovers new repos (up to 20 rounds) with all previously seen repos excluded

> **Per-Repo Failure Isolation:** The 3-phase parallel architecture ensures that a failure in any individual repo (clone, YAML, or PDF) does not block processing of other repos. Each phase collects successes and passes them to the next phase, while failures are logged and reported in the final summary table.

7. Logging & Monitoring

The pipeline provides comprehensive observability through dual-output logging and structured summary tables at every stage.

7.1 Dual Output Logging

- **Console (stdout):** Real-time progress visible in terminal
- **Log file:** Timestamped file at `logs/pipeline_YYYYMMDD_HHMMSS.log` — one file per run, UTF-8 encoded

Log format: `2026-03-26 14:30:15 [INFO] ✓ PASS: openai/whisper (★ 96592)`

7.2 Progress Tracking

- **Per-stage counters:** `[45/109]` format shows progress within each filter/phase
- **Milestone markers:** Checkmark tables (✅/❌) after Discovery, Filter 1, Filter 2
- **Phase banners:** `═══ PHASE 2 — YAML generation for 84 repos (6 workers) ═══`

7.3 Final Summary Tables

At pipeline completion, three ASCII tables are logged:

Pipeline Steps Table

```
+---------------------+--------------------------------------------+-----
---+
| Step                | Result                                     |
Status |
+---------------------+--------------------------------------------+-----
---+
| Discovery           | 84 repos accepted                          | ✅
|
| Clone + YAML + PDF  | 84 processed, 0 skipped, 0 failed          | ✅
|
| repos.yml append    | 84 entries added                           | ✅
|
+---------------------+--------------------------------------------+-----
---+
```

Final Summary Table

```
+----------------------+-------+
| Metric               | Count |
+----------------------+-------+
| Total repos accepted | 84    |
| Cloned               | 84    |
| YAMLs generated      | 84    |
| PDFs generated       | 81    |
| BZ2 compressed       | 81    |
| Added to repos.yml   | 84    |
| Drive PDFs uploaded  | 81    |
| Drive BZ2s uploaded  | 81    |
| Drive JSONL uploaded | ✅    |
+----------------------+-------+
```

Per-Repo Status Table

```
+--------------------------+-------+------+-----+-----+-----------+-----
--+
| Repo                     | Clone | YAML | PDF | BZ2 | repos.yml |
Drive |
+--------------------------+-------+------+-----+-----+-----------+-----
--+
| openai/whisper           | ✅    | ✅   | ✅  | ✅  | ✅        | ✅
|
| fastapi/fastapi          | ✅    | ✅   | ✅  | ✅  | ✅        | ✅
|
| pallets/flask            | ✅    | ✅   | ✅  | ✅  | ✅        | ✅
|
| ...                      | ...   | ...  | ... | ... | ...       | ...
|
+--------------------------+-------+------+-----+-----+-----------+-----
--+
```

8. Configuration Reference

All configurable parameters are defined as module-level constants near the top of the script. No external configuration files, environment variables (other than `GITHUB_TOKEN`), or command-line arguments are required.

8.1 Pipeline Behavior

Constant	Default	Description
`TARGET_REPO_COUNT`	1,000	Number of repositories to discover per round
`MIN_FILTER2_PASS`	3	Minimum repos that must pass Filter 2 before proceeding
`MAX_RETRY_ROUNDS`	20	Maximum discovery retry rounds
`MAX_CRAWL_PAGES`	30	Maximum pages crawled per documentation site

8.2 Concurrency Settings

Constant	Default	Description
`FILTER_WORKERS`	8	Parallel threads for Filter 1 & Filter 2 API checks
`CLONE_WORKERS`	8	Parallel git clone operations (network-bound)
`YAML_WORKERS`	6	Parallel YAML metadata generation (GitHub API calls)
`PDF_WORKERS`	3	Parallel Playwright browser instances (memory-heavy)
`UPLOAD_WORKERS`	5	Parallel Google Drive file uploads

8.3 Quality Thresholds

Constant	Default	Description
`PRIMARY_STAR_THRESHOLD`	5,000	Star count for SHOULD score in Filter 2
`FALLBACK_STAR_THRESHOLD`	3,000	Minimum stars for MUST check in Filter 1
`MIN_PYTHON_RATIO`	0.95	Minimum Python code ratio by bytes (95%)
`MAX_REPO_SIZE_KB`	500,000	Maximum repository size in KB (500 MB)

9. Prerequisites

Requirement	Details	Required?
Python 3.10+	Any standard CPython installation. The script creates its own venv.	REQUIRED
Git	Must be available on `PATH` for repository cloning.	REQUIRED
Internet Connection	Required for GitHub API, git clone, Playwright browser download, and Drive upload.	REQUIRED
`GITHUB_TOKEN`	Set as environment variable. Provides 5,000 requests/hour vs. 60 without. Generate at github.com/settings/tokens.	RECOMMENDED
`credentials.json`	OAuth2 client secret from Google Cloud Console (APIs & Services → Credentials → OAuth 2.0 Client IDs). Required for Drive upload only.	OPTIONAL

Quick Start

```
# Set GitHub token for higher rate limit
export GITHUB_TOKEN="ghp_your_token_here"

# Place credentials.json in same directory (for Drive upload)
# Then simply run:
python commit0_automation.py

# Everything else is automatic — venv, deps, browser, pipeline, upload
```

Zero Configuration Required: The only mandatory prerequisite is Python 3.10+ and Git. The script self-provisions everything else on first run, including the virtual environment, all Python packages, and the Chromium browser binary.

10. Output Files

10.1 Local Artifacts

File / Directory	Description
`raw_repos.jsonl`	All discovered repositories — one JSON object per line. Used for re-run exclusion.
`filtered_stage1.json`	Repositories that passed Filter 1 (intermediate cache).
`final_selected.json`	Final selection: repositories that passed both filters.
`processed_repos.json`	Processing state tracking — enables resume on re-runs. Tracks YAML path, PDF status, and next index.
`build_dataset/data/*.yml`	Individual YAML metadata files per repository (name, tag, python version, install cmd, test cmd, etc.).
`build_dataset/data/repos.yml`	Combined YAML file with all processed repo entries, indexed sequentially.
`build_dataset/pdfs/{repo}/`	Per-repository PDF directory containing `{repo_name}.pdf.bz2` (compressed documentation).
`logs/pipeline_*.log`	Timestamped execution logs (one file per run). Full pipeline output with UTC timestamps.
`drive_token.json`	Cached OAuth2 token for Google Drive (auto-refreshed on subsequent runs).
`.venv_commit0/`	Self-managed virtual environment with all dependencies.

10.2 Google Drive Structure

```
Automation Scripts Data/
└── 26 March 2026/               ← Date-stamped folder per run
    ├── Raw Data/
    │   └── raw_repos.jsonl       ← All discovered repos
    └── PDF/
        ├── openai_whisper/
        │   └── openai_whisper.pdf.bz2
        ├── fastapi_fastapi/
        │   └── fastapi_fastapi.pdf.bz2
        ├── pallets_flask/
        │   └── pallets_flask.pdf.bz2
        └── ... (one folder per repo)
```